

From Autonomous AI to Production-Grade Agent Systems

Human-in-the-Loop, SOP-managed context, and sandboxed execution for production-grade agent systems





郑立（布拿拿）

Dify Coder, Dev&Rel, Issue & PR terminator.



6,423 contributions in 2025 in langgenius

Contribution settings ▾

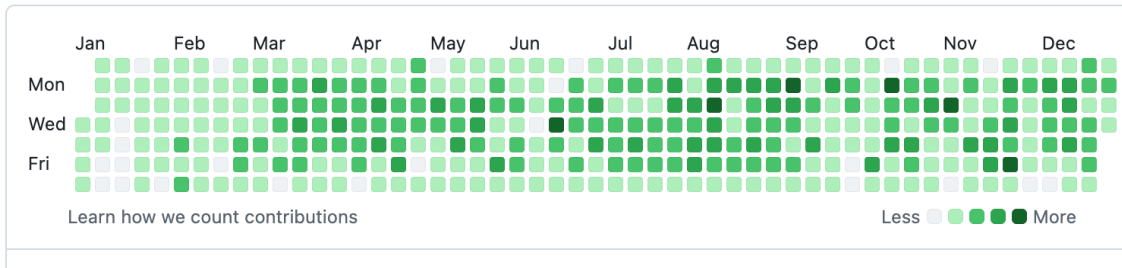
2026

2025

2024

2023

2022



Agent 已经能行动了——但生产环境还没准备好

模型能力已经跨过了“能用”的门槛，但真正把 Agent 推进生产时，团队撞上的往往是这三面墙。



幻觉直达客户

没有人工关卡，AI 生成的错误内容可以直接触达终端用户，一次事故就能摧毁信任。



合规无法闭环

金融、医疗、政务场景需要审批记录和可追溯链路，纯自动化流程过不了审计。



工具链脆弱

Prompt 越塞越大、工具越加越多、交接全靠隐式状态——维护成本远超预期。



今天要聊的，就是 Dify 如何用三项架构演进补上这些缺口。



Content 目录

01 演进之路

02 Human-in-the-Loop

03 Agent-x Skills

04 沙箱运行时 & 协作开发



Part 01

演进之路

从 Prompt 管道到 Agent 编排，为什么这次变化值得重视



LLM 应用的三代演进

每一代都释放了新的价值 —— 也带来了新的复杂性。

第一代

Prompt → 响应

单轮补全。无记忆、无工具、无状态。

ChatGPT 封装

单次摘要

第二代

管道编排

节点链式调用，支持数据转换、RAG 与条件分支。

LangChain

Dify Workflow

第三代 · 当下

Agent 系统

该停的时候停下来等人判断，该复用的时候调 Skills 和 SOP，该交接的时候把结果显式传下去。

HITL

Skills

沙箱



三大架构演进

决定 Dify 能否走向生产级 Agent 的三项关键能力。



Human-in-the-Loop 节点

把暂停、审核和恢复直接做进工作流本身，让 AI 在合规场景里也能稳稳落地。



Agent × Skills

让 Agent 回到“做判断”的位置，把 SOP、Skills 和可交接结果拆出来，形成更清晰的分工。



沙箱运行时 & 协作开发

给 Agent 一个真实可用但边界清晰的运行环境，同时把共享知识和团队协作重新带回工作流里。



Part 02

演进之路

从 Prompt 管道到 Agent 编排，为什么这次变化值得重视



为什么团队需要 HITL

最好的自动化，不是完全无人参与，而是只在人真正该介入的时刻介入。



目标会变化

工作进行到一半时，优先级和上下文可能会变，没有暂停点，流程就会变得僵硬。



信任仍然缺失

高风险流程里，团队需要看得见的检查点，才敢让 AI 代表业务继续往下走。



集成过于复杂

外部审批队列和 Webhook 会把人工审核变成额外工程，而不是原生能力。



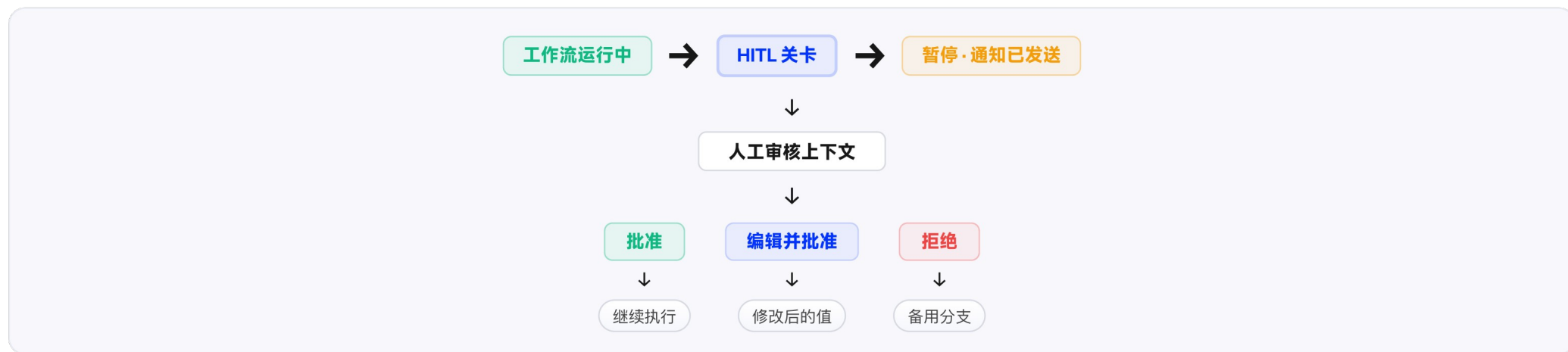
Human Input 节点

在图里暂停、拿到人的判断，然后沿正确分支原生恢复执行。



Human Input 节点怎么工作

流程先暂停，把该看的上下文交给人，再按三种结果之一继续执行。



📩 送达

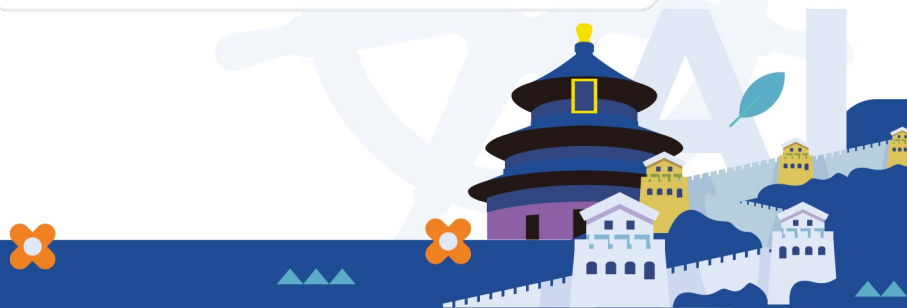
生成审核页并送达正确的人。

📄 变量

插入可编辑字段，安全回传新值。

🔧 动作

按钮、分支和超时规则，确保恢复。



HITL 该放在哪些地方

好的 HITL 节点应该落在关键决策点上，而不是铺满整条流程。

外部动作之前

在发送邮件、发布内容、提交工单或触达客户之前暂停一次。

置信度下降时

在异常、边界情况或低置信结果出现时再人工介入，而不是每次都审。

上下文缺失时

只补一个关键字段，然后让流程带着新值继续自动执行。

策略要求签核时

财务、合规和对客流程通常都需要一个看得见的审批点。



节点不一定要多，位置放对通常更重要。如果审核者还需要打开另一个系统才能完成任务，说明设计还没完成。



梁·投资服务团队

当自动化结果要直接面向客户时，HITL 能把专家判断放在最关键的位置上。

扩张难题

40 分钟

每位客户的人工处理时间

100+

需要服务的客户数

报告生成已经自动化，但财务更新真正发给客户前，仍然需要合规团队做最后确认。

HITL 放置位置

汇总之后

异常出现时

发送之前

审核者看到的就是客户最终会看到的内容，必要时编辑后再一键批准。到六月，100 位客户都收到了一致的高质量报告。

“系统负责计算，人类在最关键的地方施加判断。”



闵·全球支持团队

HITL 不只是用来审批，它也很适合在流程中补齐缺失信息。

支持难题

员工要在 HR、财务和 IT 的不同门户间来回切换。很多请求一开始就缺少路由所需的关键信息。

统一入口

问题分类

知识路由

HITL 怎么发挥作用

当研发部 Jason 询问报销时， workflow 发现缺少地点信息，于是通过 Human Input 节点补采，随后返回了正确的上海办公室政策。

“整个体验更像是人和 AI 在协同工作。”

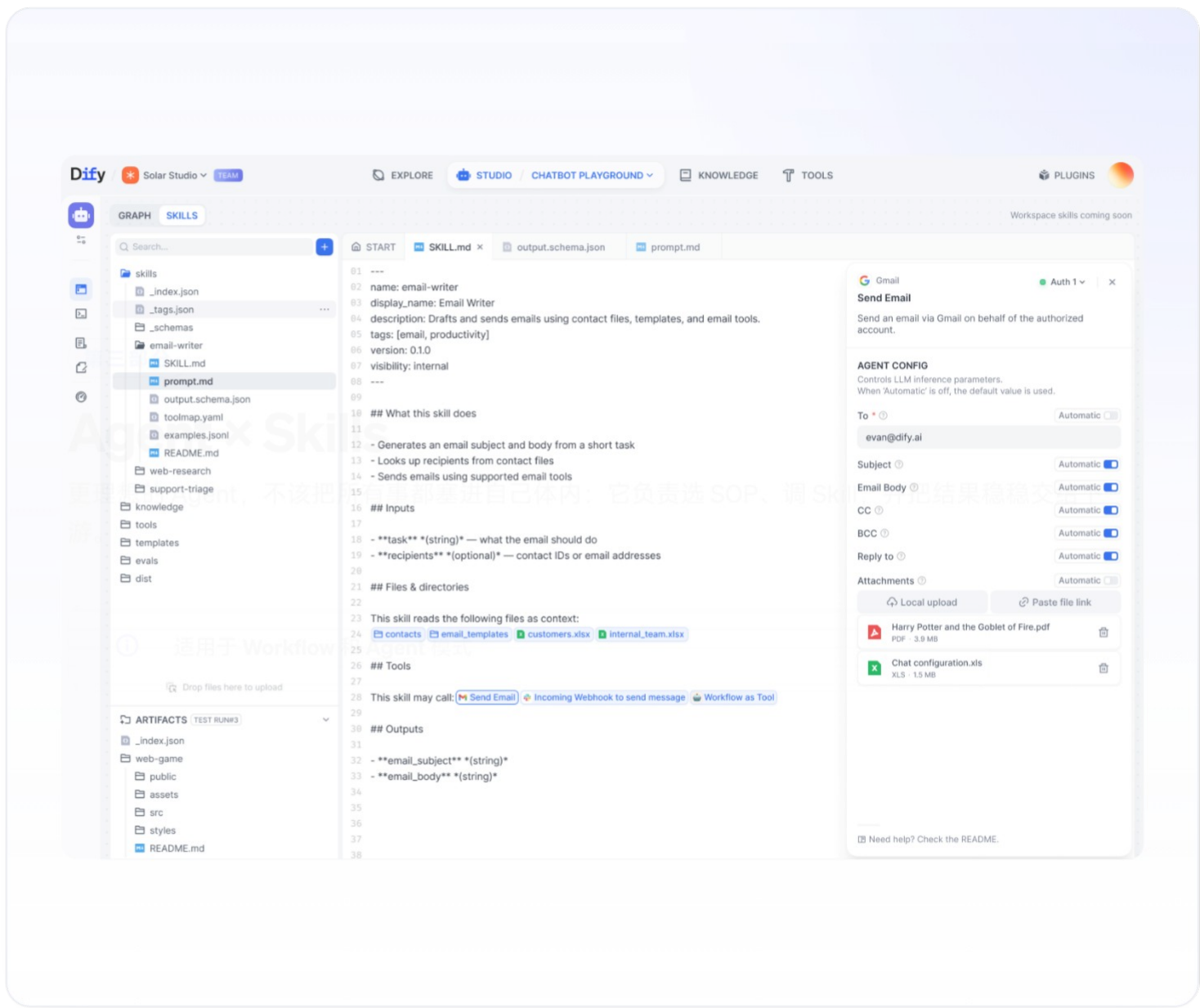


Part 03

Agent × Skills

现状问题、Skill、SOP 上下文，以及可交接的产物





心智模型

更轻的 Agent，更强的交接

Agent 负责选 SOP 和 Skill，执行沉到可复用单元里， workflow 拿到的是可以稳定路由的交付物。

AGENT 负责

目标 选择 SOP 调用 Skills 决定下一步

WORKFLOW 拿到

文本 文件 结构化字段 记忆快照

运营上的收益

工具菜单更干净，文件流更清楚，也不用把同样的 Prompt 逻辑复制进多个 workflow 里。



为什么今天的 Agent 仍然不够好

今天最成熟的团队，还是经常得绕开运行时，自己再补一层。



Agent 流程仍偏单次执行

多轮行为和多 Agent 交接，在 Workflow 里仍然不够自然。



工具菜单制造噪音

工具一多，Prompt 空间就被挤占，工具路由也会越来越脆弱。



文件交接很脆弱

文件经常依赖隐藏标识符传递，而不是依赖直观的运行时工件。



调试链路太长

策略、工具路由、文件流和记忆分散在多层，出错时很难定位。



从一个大 Prompt 到轻量编排层

Agent 应该负责做判断，而不是把所有执行细节都塞进 Prompt 里。

之前 —— 一个大 PROMPT 包打天下

全都塞在 PROMPT 里

工具路由

文件处理

重试逻辑

输出格式

容易出的问题

逻辑重复

难以测试

工具膨胀

状态隐藏

之后 —— AGENT 只做编排

AGENT 负责

目标

选择 SOP

调用 Skills

挑选交付物

WORKFLOW 拿到

文本

文件

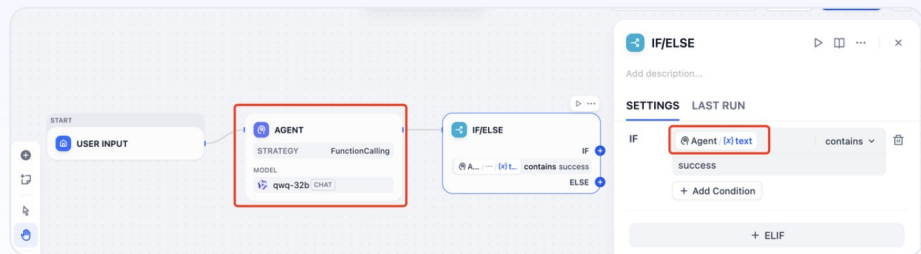
结构化字段

记忆快照



交付物不清楚，Workflow 就跑不顺

如果真正有价值的产物还留在 Agent 的内部记忆里，下游节点就只能从自然语言里猜。



示例：IF/ELSE 节点只能从文本里推断状态。

文本不是状态

靠判断 Agent 有没有说出 `success`，既脆弱，也难维护。

文件会消失

原始表格、报告或中间文件可能埋在记忆里，下游只看到总结。

AGENT 无法接力

后一个 Agent 不能稳定知道前一个 Agent 究竟交付了什么。



一个节点到底该交付什么

生产级工作流需要的，不只是一个润色后的答案。

文本回答

给人看的最终解释或回复。

文件

报告、表格、图片等，可继续被下游节点使用。

结构化字段

状态、决策、ID、参数等，供分支或工具直接读取。

记忆快照

让后续节点可以从中提取事实、参数甚至文件。



如果下游节点接不住，它就不算真正交付出来。



什么是 Skill?

Skill 可以理解成可复用的执行单元：把 SOP、执行逻辑和稳定的交接格式封装在一起。



自带 SOP

“这类事该怎么做”不再散落在各个节点里，而是跟着 Skill 一起沉淀下来。



可复用

发布一次，就可以被不同 Agent 和 Workflow 反复调用。



可测试

用固定输入单独运行 Skill，而不必触发整条流程。



可锁定版本

Agent 可以锁住稳定版本，不必每次都被共享 Skill 的更新牵着走。

典型输入来源

对话上下文

前置节点输出

文件

记忆提取



一套 SOP，多种入口

Context Engineering 需要一个团队共用的落点，而不是到处复制 Prompt。

今天 —— SOP 散落在各个节点里

同一 SOP 重复写

难以评审

最佳实践漂移

更好的方式 —— 共用的 `/sops` 工作区

写一次

按入口文件区分

随工作流版本化



很多 Agent 其实共用的是同一套 SOP，只是入口文件不同、关注点不同。



实战：从散落的 Prompt 到统一 Skill 库

一个电商运营团队把 5 条独立工作流里重复的客服 SOP 收敛成一个共享 Skill，维护量降了四倍。

之前 —— 5 条流程各写各的

退换货流程

物流查询流程

售后投诉流程

订单异常流程

VIP 服务流程

每条流程里都有一段近似的"客服话术 SOP"和"工单分类逻辑"，改一处要改五处。

之后 —— 一个 SKILL，五个入口

共享 SOP：客服话术

共享 SOP：工单分类

每条流程只定义自己的入口文件和特有逻辑，共享部分由 Skill 统一维护和版本化。

“以前改一个话术要开五个 Workflow 逐个找，现在改一次 Skill 全部生效。”



Skill + SOP 驱动的 Agent 架构

推理层尽量轻，执行层围绕文件、命令和可复用产物展开。

输入

- 用户请求
- 前置节点输出
- 上传文件

AGENT 层

- 选择 SOP
- 组装上下文
- 调用 Skills
- 决定下一步

运行时工作区

- /sops
- 命令
- 文件
- 版本化 Skills

可交接结果

- 文本
- 文件
- 字段
- 记忆
- HITL

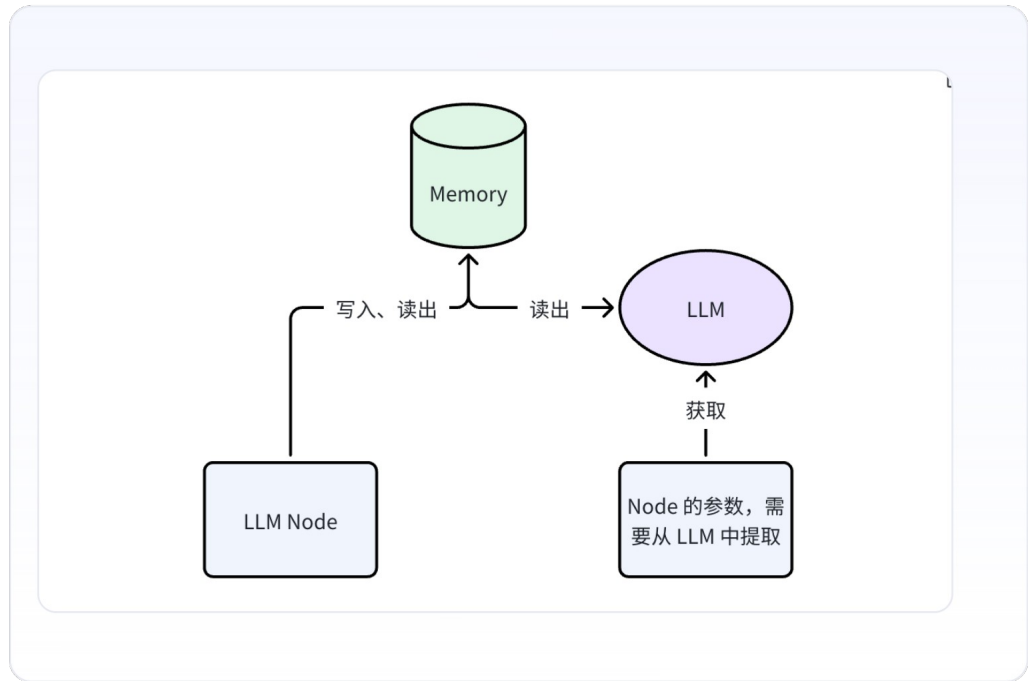


更好的心智模型是 SOP、文件和产物，而不是一个越来越大的工具下拉框。



记忆提取让上下文可复用

Memory 不再只是实现细节，而是可以继续往下传的工作流产物。



④ 成本与延迟

提取 LLM 调用很轻量：读取有界上下文窗口，输出结构化字段。典型开销 <1s、<500 tokens。

🛡️ 提取失败的 FALLBACK

如果提取失败，节点回退到上游 Agent 的原始文本输出，工作流不会静默中断。

④ 与 RAG 的区别

RAG 从外部语料库检索；Memory Extraction 从同一次运行的工作上下文中提取。无需向量库 —— 这是工作流内状态，不是跨会话检索。

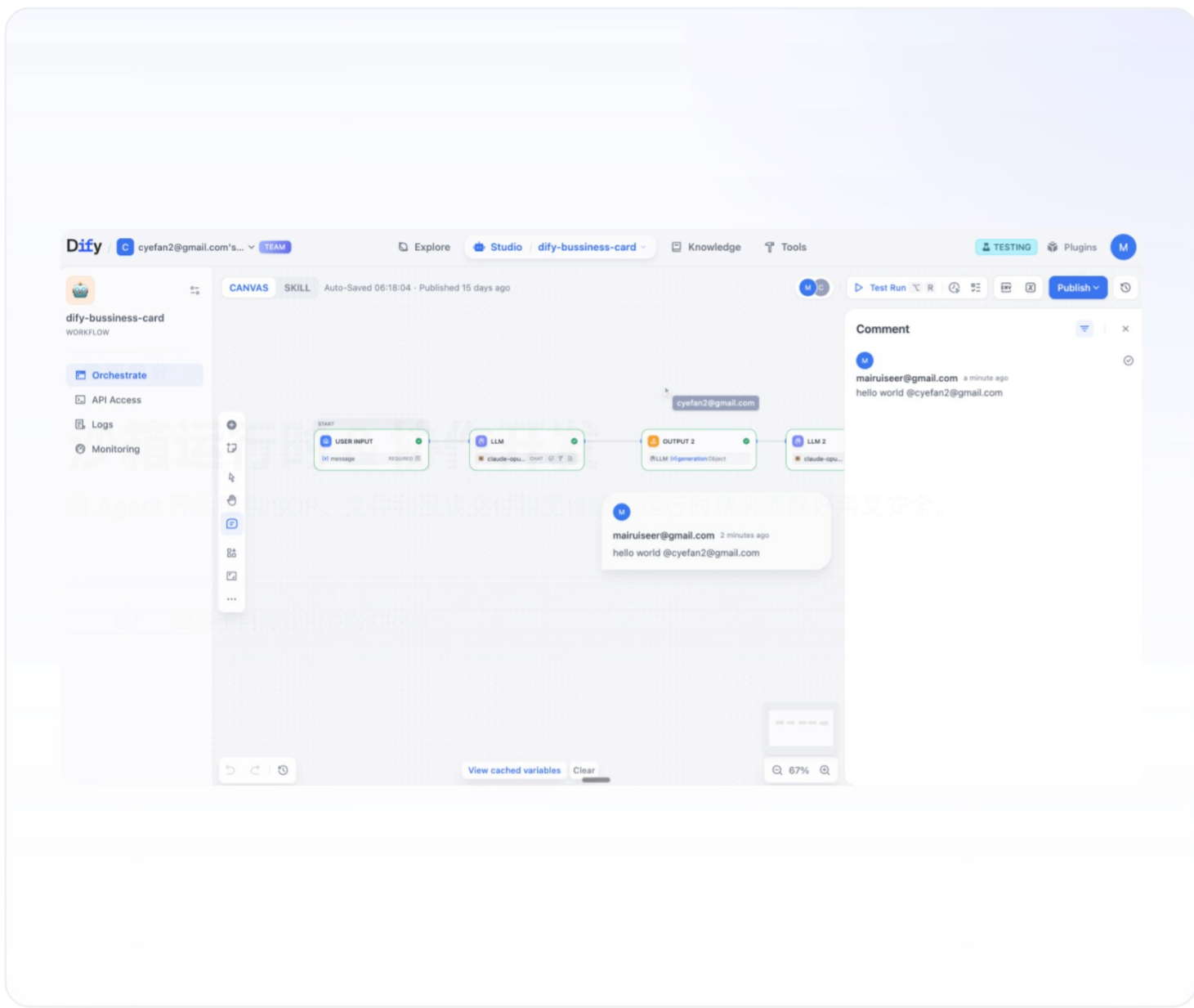


Part 04

沙箱运行时 & 协作开发

POSIX 风格运行时、Command 节点、安全执行与团队协作





生产表面

好用的运行时，共同维护的资产

命令、文件和 SOP 只有在运行时既安全、可追踪，又方便团队共审时，才会真正变成团队资产。

运行时原语

命令 文件 stdout POSIX 工作区

安全 + 可追踪

无主机访问 资源限制 trace 日志 错误回放

团队生命周期

草稿、评审、发布和版本历史，把“个人实验”变成团队可共同运营的工作流。



Command 节点：简单，但很有用

输入一行命令，返回 stdout，其余产物留在运行时里交给下一步。

示例

```
report --input ./turnsheet.csv --format json
```

命令行输入

stdout 输出

文件留在运行时

对模型更自然

模型在预训练里已经见过大量命令、管道和文件路径。

产品表面更简单

不需要为每个小转换动作都单独设计一套工具 UI。

交接更干净

大的工件保留成文件，显式地往下传，而不是硬塞回 Prompt。



从工具列表到 POSIX 式工作区

不要把每种能力都包成一张 Tool 卡，而是直接把命令、文件和 stdout 暴露给运行时。

之前 —— 以 TOOL 为中心的编排

```
step1: A = google_search(query="Dify", max_size=1000)
step2: B = summary(query=A)
```

类型转换隐藏

输出停在内存里

每个工具都要做 UI

之后 —— POSIX 式执行

```
summary --query "$(google_search --query dify --max_size 1000)"
```

统一字符串接口

shell 原生组合

先 ls /bin 认识环境



不受约束的代码有什么风险

Agent 需要真实可用的执行环境，但不该直接碰到宿主机。



宿主系统访问

没有隔离时，代码可以读取本地凭证、环境变量和文件。



没有资源边界

失控循环或内存暴涨会拖垮共享运行时上的所有任务。



供应链风险

导入的第三方包可能静默外传 workflow 数据，除非环境被严格控制。



沙箱不是为了削弱能力，而是把能力稳稳地关在安全边界里。



沙箱代码执行

给 Agent 一个像小型 Unix 工作站一样可用的环境，同时放在严格的安全边界里。

安全边界

- ☒ 无法访问宿主文件系统
- ☒ 网络受白名单限制
- ☒ 每次运行都有 CPU / 内存上限
- ☒ 每个节点可配置超时

可用的运行时能力

ls /bin stdin/stdout I/O 文件作为交接 Python 3.11+
JavaScript (Node 20) 外置文件存储

如何启用

Cloud 默认开启 自托管设置 `SANDBOX=true`



可观测性：让每一步都可追溯

生产级系统不只要能跑通，还要在出问题能快速定位、在日常运行中能持续度量。



节点级追踪

每个节点的输入、输出、耗时和 Token 消耗都有独立追踪记录，出问题时可以精确回溯到具体步骤。



成本追踪

按工作流、按节点、按模型拆分 Token 成本，让团队知道钱花在了哪里。



延迟分析

瓶颈在推理、工具调用还是文件 I/O？延迟分布图让优化有据可依。



错误重放

失败的运行可以连同完整上下文一起重放，不用猜、不用复现。



协作 workflow 开发

workflow 本身会变成团队共同维护的产品资产。



角色访问控制

不同成员可以分别起草、审查、发布，而不会互相覆盖。



版本历史

每次发布都会形成快照，便于对比和快速回滚。



草稿 → 评审 → 发布

流程生命周期变得清晰可重复，而不是散落在截图和聊天记录里。



共享 SOP 资产

最佳实践不再是个人 Prompt 片段，而会沉淀成团队资产。

一个典型协作流程

有人起草 workflow，有人评审 SOP，负责人发布通过的版本，整个过程都留有历史记录。



融合全景

一个真正能进生产的 Agent 系统里，推理、执行和人工审核都围绕显式交付物协同运转。

输入 → AGENT 推理层

用户查询 / 文件 / 定时触发 选择 SOP 调用 Skills 组装上下文

执行层（沙箱）+ HITL 关卡

Command 节点 Skills 文件 + stdio 代码沙箱 暂停 → 审核 → 恢复

交付结果 + 可观测性

文本 / 文件 / 结构化字段 记忆快照 追踪日志 成本追踪



生产级系统的关键，就是每一步都要给下一步留下真正接得住的产物。

全球社区数据

开源生态驱动，GitHub Top 100 项目

GitHub Top 100 · 开源 LLM Ops

1M+

POWERED BY DIFY

130K+

GITHUB STARS

150+

覆盖国家

1,000+

开源贡献者

60+

行业应用

550M+

总下载量

Trusted by Leaders Across Industries Worldwide

 BeiGene

 ThermoFisher
SCIENTIFIC

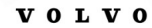


 MAERSK

 NOVARTIS











 BOSCH



 DECATHLON





 MUFG





 Panasonic





 NTT EAST



 JEP
日本電子計算株式会社







下一步

不需要一次用齐三个能力 —— 挑一个最痛的场景，今天就可以开始。



试用 HITL 节点

在 Dify 最新版中拖入一个 Human Input 节点，给你的 workflow 加上第一个人工关卡。

现已可用



探索 Agent Skills

把你最常复制的 SOP 提炼成第一个 Skill，体验复用和版本化带来的效率提升。

即将推出



加入社区

Star 项目、加入 Discord，和全球开发者一起塑造 Agent 系统的未来。

langgenius/dify



Thanks.

